Gebze Technical University

CSE344 SYSTEM PROGRAMMING

-

HW3 REPORT

Veysel Cemaloğlu

200104004107

Problem Overview

This project implements a concurrent parking lot management system utilizing semaphores and shared memory. The system facilitates the arrival, and subsequent transfer of automobiles and pickups to designated parking spaces within the main lot. This design ensures proper synchronization and efficient resource allocation.

Two distinct attendant processes, coordinated through semaphores, manage the overall parking operation. Semaphores act as a critical synchronization mechanism, guaranteeing conflict-free processing of incoming vehicles and preventing race conditions that could arise from concurrent access to shared resources.

Compiling and Running

Compile the code:

`make`

Run the program:

`make run`

Clean the object files:

`make clean`

IMPORTANT:
Program will run until there is no available parking lot.

Key Features of the Parking Lot Management System

This system utilizes semaphores and shared memory to manage a parking lot with dedicated spaces for automobiles and pickups. It prioritizes order and resource management through concurrent processing.

Vehicle Handling

Vehicles arrive at random intervals and are categorized as either automobiles or pickups upon arrival. For orderly processing and to prevent conflicts, the system handles only one vehicle at a time.

Vehicle Parking

The system designates a parking lot with eight spaces for automobiles and four spaces for pickups. To ensure efficient management, each vehicle type is assigned a dedicated attendant process. This attendant is responsible for overseeing the parking of its designated vehicle type (automobiles or pickups) within the parking lot.

Synchronization Using Semaphores

Semaphores act as a control mechanism, ensuring that vehicle owners can only park if there are available spots. They also synchronize the attendants, allowing for a smooth transfer of vehicles from the parking lot.

Shared Resource Protection

Shared variables track the availability of parking spots. Semaphores are crucial in protecting these variables from race conditions that could arise from concurrent access by multiple attendants. Additionally, semaphores manage the ensure single-vehicle entry at a time.

Concurrency Management

The system leverages concurrency through the use of threads. These threads represent two distinct entities:

- **carOwner:** Threads simulate the arrival of vehicles, categorized as automobiles or pickups.
- **carAttendant:** Threads represent the attendants responsible for managing each vehicle type within the parking lot.

Semaphores play a vital role in this concurrent environment. They act as synchronization primitives, ensuring orderly processing and data integrity.

SOLUTION APPROACH

1. Initialization of Semaphores and Shared Variables:

- **Semaphores**: We use semaphores to control access to shared resources and to synchronize the threads.
  - `newPickup` and `inChargeforPickup`: These semaphores manage the arrival and parking of pickup vehicles.
  - `newAutomobile` and `inChargeforAutomobile`: These semaphores manage the arrival and parking of automobile vehicles.
- **Shared Variables**:
  - `mFree_automobile`: Counter to track the number of free spots available for automobiles.
  - `mFree_pickup`: Counter to track the number of free spots available for pickups.

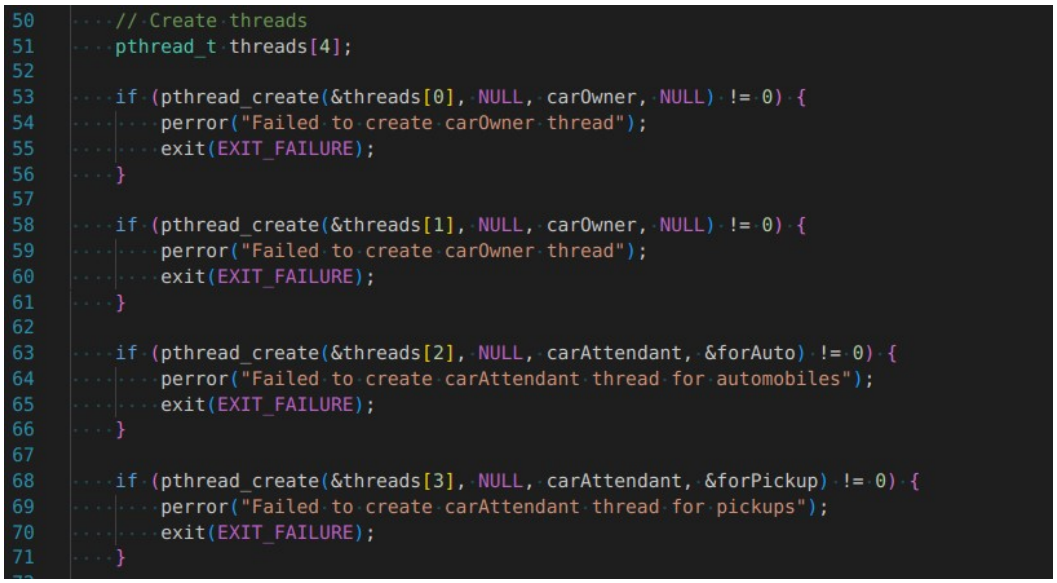
Initialization involves setting these semaphores to 0 initially, which ensures that the attendant waits until a vehicle arrives. The counters are set to the maximum capacity of the respective parking areas.

```
8   #define MAX_AUTOMOBILE 8
9   #define MAX_PICKUP 4
10
11  // Semaphores for synchronizing vehicle arrivals and parking attendants
12  sem_t newPickup, inChargeforPickup;
13  sem_t newAutomobile, inChargeforAutomobile;
14
15  // Counters for parking space availability
16  int mFree_automobile = MAX_AUTOMOBILE;
17  int mFree_pickup = MAX_PICKUP;
18
```

```
26      // Initialize semaphores
27      if (sem_init(&newPickup, 0, 0) != 0) {
28          perror("Failed to initialize newPickup semaphore");
29          exit(EXIT_FAILURE);
30      }
31
32      if (sem_init(&inChargeforPickup, 0, 1) != 0) {
33          perror("Failed to initialize inChargeforPickup semaphore");
34          exit(EXIT_FAILURE);
35      }
36
37      if (sem_init(&newAutomobile, 0, 0) != 0) {
38          perror("Failed to initialize newAutomobile semaphore");
39          exit(EXIT_FAILURE);
40      }
41
42      if (sem_init(&inChargeforAutomobile, 0, 1) != 0) {
43          perror("Failed to initialize inChargeforAutomobile semaphore");
44          exit(EXIT_FAILURE);
45      }
```

2. Thread Creation:

- Four threads are created in the main function:
  - Two threads represent vehicle owners.
  - Two threads represent parking attendants.
- `pthread_create()` is used to create these threads, specifying the appropriate functions (`carOwner()` and `carAttendant()`) and passing necessary arguments.



```
50    // Create threads
51    pthread_t threads[4];
52
53    if (pthread_create(&threads[0], NULL, carOwner, NULL) != 0) {
54        perror("Failed to create carOwner thread");
55        exit(EXIT_FAILURE);
56    }
57
58    if (pthread_create(&threads[1], NULL, carOwner, NULL) != 0) {
59        perror("Failed to create carOwner thread");
60        exit(EXIT_FAILURE);
61    }
62
63    if (pthread_create(&threads[2], NULL, carAttendant, &forAuto) != 0) {
64        perror("Failed to create carAttendant thread for automobiles");
65        exit(EXIT_FAILURE);
66    }
67
68    if (pthread_create(&threads[3], NULL, carAttendant, &forPickup) != 0) {
69        perror("Failed to create carAttendant thread for pickups");
70        exit(EXIT_FAILURE);
71    }
```

3. Vehicle Owner Logic:

- The `carOwner()` function simulates vehicle owners arriving at the parking lot.
- Each owner checks if there are free spots available:
  - If an automobile owner finds a free spot, they signal the `newAutomobile` semaphore and wait for the `inChargeforAutomobile` semaphore to be posted by the attendant.
  - If a pickup owner finds a free spot, they signal the `newPickup` semaphore and wait for the `inChargeforPickup` semaphore.
- If no spots are available, the owner leaves and checks if both types of spots are full. If so, the program exits.

4. Attendant Logic:

- The `carAttendant()` function simulates attendants managing the parking of vehicles.
- Each attendant waits for a vehicle to arrive by waiting on the appropriate semaphore (`newAutomobile` or `newPickup`).
- Once a vehicle arrives, the attendant:
    - Decrements the available spot counter.
    - Simulates parking the vehicle by sleeping for a random amount of time.
    - Signals the vehicle owner by posting the `inChargeforAutomobile` or `inChargeforPickup` semaphore.

5. Thread Synchronization and Completion:

- Synchronization is achieved using semaphores to ensure proper sequence of actions between vehicle owners and attendants.
- The main thread waits for all threads to complete using `pthread_join()`.
- Upon completion, semaphores are destroyed to free resources.

```
73      // Join threads
74      for (int i = 0; i < 4; i++) {
75          if (pthread_join(threads[i], NULL) != 0) {
76              perror("Failed to join thread");
77              exit(EXIT_FAILURE);
78          }
79      }
80
81      // Destroy semaphores
82      if (sem_destroy(&newPickup) != 0) {
83          perror("Failed to destroy newPickup semaphore");
84      }
85
86      if (sem_destroy(&inChargeforPickup) != 0) {
87          perror("Failed to destroy inChargeforPickup semaphore");
88      }
89
90      if (sem_destroy(&newAutomobile) != 0) {
91          perror("Failed to destroy newAutomobile semaphore");
92      }
93
94      if (sem_destroy(&inChargeforAutomobile) != 0) {
95          perror("Failed to destroy inChargeforAutomobile semaphore");
96      }
```

TEST CASES

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/HW3$ make run
gcc -Wall -Werror -pthread -c parking_lot.c
gcc -Wall -Werror -pthread -o parking_lot parking_lot.o
./parking_lot
Automobile owner arrives. Available automobile spots: 8
Automobile attendant parks a car. Remaining automobile spots: 7
Pickup owner arrives. Available pickup spots: 4
Pickup attendant parks a pickup. Remaining pickup spots: 3
Automobile owner arrives. Available automobile spots: 7
Automobile attendant parks a car. Remaining automobile spots: 6
Pickup owner arrives. Available pickup spots: 3
Pickup attendant parks a pickup. Remaining pickup spots: 2
Automobile owner arrives. Available automobile spots: 6
Automobile attendant parks a car. Remaining automobile spots: 5
Pickup owner arrives. Available pickup spots: 2
Pickup attendant parks a pickup. Remaining pickup spots: 1
Automobile owner arrives. Available automobile spots: 5
Automobile attendant parks a car. Remaining automobile spots: 4
Automobile owner arrives. Available automobile spots: 4
Automobile owner arrives. Available automobile spots: 4
Automobile attendant parks a car. Remaining automobile spots: 3
Pickup owner arrives. Available pickup spots: 1
Pickup attendant parks a pickup. Remaining pickup spots: 0
Automobile attendant parks a car. Remaining automobile spots: 2
Pickup owner leaves. No available parking spots.
Pickup owner leaves. No available parking spots.
Pickup owner leaves. No available parking spots.
Pickup owner leaves. No available parking spots.
Automobile owner arrives. Available automobile spots: 2
Automobile attendant parks a car. Remaining automobile spots: 1
Pickup owner leaves. No available parking spots.
Automobile owner arrives. Available automobile spots: 1
Automobile attendant parks a car. Remaining automobile spots: 0
Automobile owner leaves. No available parking spots.
No available parking spots. Exiting program.
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/HW3$
```

```
./parking_lot
Automobile owner arrives. Available automobile spots: 8
Pickup attendant parks a pickup. Remaining pickup spots: 3
Automobile attendant parks a car. Remaining automobile spots: 7
Pickup owner arrives. Available pickup spots: 4
Automobile owner arrives. Available automobile spots: 7
Automobile attendant parks a car. Remaining automobile spots: 6
Automobile owner arrives. Available automobile spots: 6
Automobile attendant parks a car. Remaining automobile spots: 5
Automobile owner arrives. Available automobile spots: 5
Pickup owner arrives. Available pickup spots: 3
Pickup attendant parks a pickup. Remaining pickup spots: 2
Automobile attendant parks a car. Remaining automobile spots: 4
Automobile owner arrives. Available automobile spots: 4
Pickup attendant parks a pickup. Remaining pickup spots: 1
Automobile attendant parks a car. Remaining automobile spots: 3
Pickup owner arrives. Available pickup spots: 2
Automobile owner arrives. Available automobile spots: 3
Automobile attendant parks a car. Remaining automobile spots: 2
Automobile owner arrives. Available automobile spots: 2
Pickup owner arrives. Available pickup spots: 1
Automobile attendant parks a car. Remaining automobile spots: 1
Pickup attendant parks a pickup. Remaining pickup spots: 0
Automobile owner arrives. Available automobile spots: 0
Automobile attendant parks a car. Remaining automobile spots: 0
Automobile owner leaves. No available parking spots.
No available parking spots. Exiting program.
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/systemprog/HW3$
```